GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 06 — Servidores concurrentes
Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Lo que veremos en este bloque

🐢 **Servidor secuencial**
Un cliente cada vez

🥶 **La espera**
Un cliente lento congela la cola

🚀 **Hilo por cliente**
threading.Thread por conexión

⚡ **ThreadPool**
max_workers + cola interna

🧭 **El camino**
Del límite lineal a la producción

Partimos del servidor de la UD 05, que atiende **de uno en uno**. En el bloque 02: **benchmark**, **Lock** para el estado compartido, límites y el **servidor concurrente completo**.

Servidor secuencial

La idea en una frase

> Un servidor **secuencial** atiende a un cliente cada vez: `accept()` recibe la conexión, la procesa hasta el final y solo entonces vuelve a aceptar. Los demás esperan en la **cola del socket**.

⚠️ Los clientes ya conectados **no ven ningún error**: la conexión TCP la acepta el **sistema operativo**. Pero tu código no los atiende: están **en cola**, invisibles para el programa.

An error occurred on this slide. Check the terminal for more information.

La traza: 3 clientes a la vez

Sé el servidor secuencial

Cliente	Conecta	Es atendido	Total de espera
Cliente-1	t = 0 s	t = 3 s	3 s
Cliente-2	t = 0,1 s	t = 6 s	~6 s
Cliente-3	t = 0,2 s	t = 9 s	~9 s

1. Los tres conectan a la vez: el SO los mete en la **cola del socket**
2. `accept()` los va **recogiendo de uno en uno**, tras cerrar al anterior
3. Resultado: el tiempo total crece **linealmente**: **n × tiempo_por_cliente**

ℹ️ Con hilos, los tres terminarían a los **3 s**: eso lo construimos en el punto 3.

El problema de la espera

La idea en una frase

> El problema del secuencial no es el número de clientes, sino el **cliente lento**: uno que tarde 3 segundos (o 30) congela a **todos** los que esperan detrás.

1. El trabajo de cada cliente (cálculo pesado, base de datos lenta, una red de 56k...) **bloquea el** `accept()`
2. El cliente más rápido del mundo espera a que termine **el más lento**
3. Una petición de 30s pone a **toda la cola** a esperar 30s... y al que llega detrás, otros 30s

La ventanilla única

La analogía del banco

🏦 **Ventanilla única (secuencial)**

10 personas en fila: la 1 tarda 3 min y las otras 9 esperan. Ya están «en el banco» (conectadas al socket), pero su trámite **no empieza**.

🏢 **Ventanillas múltiples (concurrente)**

Cada persona tiene su ventanilla: **todas avanzan a la vez**. Diez ventanillas= diez clientes atendidos en paralelo.

Situación	Secuencial	Concurrente
10 clientes, 2s cada uno	~20 segundos	~2 segundos
1 cliente lento de 30s	Todos esperan +30s	Solo el lento tarda 30s
Picos de conexiones	La cola crece sin límite visible	Se atienden en paralelo

Mini-chequeo: secuencial y espera

🧠 Responde en 30 segundos:

1. ¿Por qué los clientes 2 y 3 «se conectan» sin error pero no reciben respuesta?
2. Si 5 clientes conectan a la vez y cada uno tarda 2s, ¿cuánto espera el quinto?
3. ¿Qué línea del código es la que se «atasca» mientras se procesa un cliente?

🔄 **Respuestas:**

- La conexión TCP la acepta el **sistema operativo** (cola del socket); el `accept()` del código no llega a ejecutarse
- Empieza a ser atendido a los **8 s** y termina a los **10 s**: n × tiempo_por_cliente
- La atención al cliente (`time.sleep(3)`): bloquea el bucle y el siguiente `accept()` no se ejecuta

Hilo por cliente

La idea en una frase

> En lugar de atender al cliente dentro del bucle, el servidor **lanza un hilo por cada cliente** y vuelve inmediatamente a `accept()`. Cada conexión tiene su propia «ventanilla».

1 **No hay trabajo en el bucle**: `accept()` → crear hilo → `start()` → vuelta a `accept()`. El bucle nunca se bloquea

2 `conn` **se pasa como argumento**: cada hilo recibe su socket y su dirección, y los libera con `with conn:`

El servidor multihilo

Hilos en paralelo, del módulo de la U03

servidor_multihilo.py — Python

```
import socket, threading, time
def atender(conn, addr):
    with conn:
        datos = conn.recv(1024)
        time.sleep(2)  # simular trabajo pesado
        conn.sendall(b"OK: " + datos)
        print(f"[-] Cliente {addr} atendido")
def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🚀 Servidor MULTIHILO: varios clientes a la vez")
        while True:
            conn, addr = srv.accept()
            hilo = threading.Thread(target=atender, args=(conn, addr))
            hilo.start()
```

Be the code: multihilo en acción

Sé el servidor. 3 clientes conectan a la vez

1. `accept()` devuelve la **conexión-1** → nace `Thread(target=atender)` → `start()`
2. El hilo-1 duerme en `time.sleep(2)` … y el hilo principal **ya está de nuevo en** `accept()`
3. Llegan las conexiones-2 y -3 → hilo-2 e hilo-3 arrancan **en paralelo** con el hilo-1
4. A los 2s, los tres clientes reciben su «OK» **a la vez**: nadie esperó en cola

ℹ️ El hilo principal **nunca se detiene**. El trabajo pesado vive en los hilos secundarios, que corren en paralelo gracias al **threading** de la U03.

Ventajas y el primer cuidado

Ventaja	Explicación
✅ Simple de escribir	Solo un `threading.Thread(target=atender, args=(conn, addr))`
✅ Aprovecha la CPU	Varios clientes procesados de verdad en paralelo
✅ El bucle no se bloquea	`accept()` vuelve al instante; nadie queda en cola

⚠️ La letra pequeña: **un hilo por cliente puede saturar el sistema con 10.000 conexiones**. Cada hilo consume memoria (pila de ejecución) y el SO debe alternar entre todos. Ahí entra el **ThreadPool** (siguiente punto).

Mini-chequeo: hilo por cliente

🧠 Responde sin mirar:

1. ¿Qué hace el hilo principal mientras un hilo secundario procesa a su cliente?
2. ¿Por qué NO hace falta `join()` en el bucle principal del servidor?
3. ¿Qué pasaría con este código si llegan 10.000 conexiones a la vez?

🔄 **Respuestas:**

- Vuelve a **accept()**: por eso el servidor nunca se bloquea y nadie espera en cola
- Un `join()` esperaría a que el hilo terminara... y **volveríamos al secuencial**. El `join()` solo se usa en el lanzador de clientes
- 10.000 hilos: sin memoria o cambio de contexto degradado. La solución es el **ThreadPool**

ThreadPoolExecutor

La idea en una frase

> Un **ThreadPoolExecutor** reutiliza un **número fijo de hilos**: si llegan más clientes que hilos, esperan en una cola interna en lugar de multiplicar los hilos.

ℹ️ El pool es un **equipo de camareros limitado**: 10 camareros para todas las mesas, no un camarero nuevo por cada mesa. «Si el pool se llena, los clientes esperan» suena a defecto, pero es la **protección** que buscamos.

⚠️ Con hilos ilimitados y 10.000 conexiones: cada hilo consume su **pila de memoria**, el SO pierde la vida en **context switches** y el proceso puede **morir sin memoria**.

El servidor con pool

El mismo servicio, con recursos bajo control

servidor_pool.py — Python

```python
import socket, concurrent.futures
MAX_HILOS = 10
def atender(conn, addr):
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK: " + datos)
def servidor_pool():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        with concurrent.futures.ThreadPoolExecutor(max_workers=MAX_HILOS) as pool:
            print(f"⚡ Servidor con Pool de {MAX_HILOS} hilos")
            while True:
                conn, addr = srv.accept()
                pool.submit(atender, conn, addr)
```

⚡ `submit()` devuelve un **Future**, pero en un servidor no lo usamos: solo nos interesa que la tarea se ejecute.

Dos piezas nuevas

max_workers y submit()

1. `ThreadPoolExecutor(max_workers=10)` crea el **equipo de hilos**. Se usa como contexto (`with ... as pool`) para que se cierre solo al salir
2. Se crea **una sola vez**, fuera del bucle `while True` : es un recurso que vive toda la vida del servidor
3. `pool.submit(atender, conn, addr)` sustituye a `Thread(...).start()` : entrega la tarea y el pool decide qué hilo libre la coge

✅ ¿El cliente-11 con todos los hilos ocupados? Su tarea **espera en la cola interna** del pool. No se crea ningún hilo nuevo.

¿Qué tamaño de pool elegir?

Reglas de oro (mediremos de verdad en el bloque 02)

Situación	Tamaño razonable
Servidor de prueba / clase	5-10 hilos
CPU intensivo (cálculo puro)	Nº de núcleos de la CPU
I/O intensivo (red, discos, APIs)	Más hilos que núcleos: los hilos esperan mucho en I/O

ℹ️ 💡 En un servidor casi todo es **espera de E/S** (recv, discos, bases de datos): por eso en producción conviene **más hilos que núcleos**. Pero no infinitos: lo vemos en la unidad de límites.

Mini-chequeo: ThreadPool

🧠 Responde en 30 segundos:

1. ¿Qué pasa con el cliente-11 si el pool tiene `max_workers=10` y todos están ocupados?
2. ¿Qué ventaja tiene reutilizar hilos frente a crear uno nuevo por cliente?
3. ¿Por qué el pool se crea fuera del bucle `while True`?

🔄 **Respuestas:**

- El `submit()` lo **encola**: espera a que se libere un hilo. No se satura el sistema
- Evita el coste de **crear y destruir hilos** constantemente: los mismos hilos se reutilizan una y otra vez
- Crearlo dentro del bucle significaría crear (y cerrar) un pool **en cada conexión**: un desperdicio enorme

Resumen del bloque 01

🐢

Secuencial
Un cliente cada vez; n × tiempo_por_cliente

🥶

Espera
El cliente lento bloquea el accept()

🚀

Hilo por cliente
Thread(target, args) + start(); paralelo real

⚡

ThreadPool
max_workers fijos + cola interna; producción

🛡️

Vocabulario
accept() · cola del socket · context switch · submit()

✅ Bloque 02: **benchmark** con lanzador masivo de clientes, **Lock** para el estado compartido, **límites y buenas prácticas** y el **servidor concurrente completo**.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //